VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI

Project on

DEEP LEARNING APPROACHES TO PREDICT AND CLASSIFY MALIGNANT TUMOR WITH LESION SEGMENTATION

Submitted by

ABHISHEK R	**4BB22CS004**
M V YASHWANTH	**4BB22CS035**
RENU PRASANNA M H	**4BB23CS405**
VIKAS S G	**4BB23CS406**

In partial fulfillment of the requirement for the award of the Bachelor Degree

In

Computer Science and Engineering

Under the Guidance of

Mrs. Kavitha C R B.E., M Tech.,

Associate Professor & HOD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Bahubali College of Engineering

Shravanabelagola-573 135

2025-26

BAHUBALI COLLEGE OF ENGINEERING

Shravanabelagola-573135

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project entitled **"DEEP LEARNING APPROACHES TO PREDICT AND CLASSIFY MALIGNANT TUMOR WITH LESION SEGMENTATION"** is work carried out by bonafied student of Bahubali College of Engineering, **RENU PRASANNA M H** USN:**4BB23CS405,** in partial fulfillment of VII Semester to award the Bachelor Degree in **Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year **2025-26**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report and deposited in the department library. The Project Report has been approved as it satisfies all the academic requirements in respect of Project work prescribed for the Bachelor of Engineering Degree.

Mrs. Kavitha C R	**Mrs. Kavitha C R**	**Dr. Sunil Kumar D**
Asso. Professor & Guide	Asso. Professor & HOD	Principal
Dept.of CS & E	Dept.of CS & E	

Name of the Examiners **Signature with Date**

1.

2.

ACKNOWLEDGEMENT

Inspiration and guidance are valuable in all aspects of life, especially what is academic. "Experience is the best teacher", is an old saying. The satisfaction and pleasure that accompany the gain of experience would be incomplete without mentioning the people who made it possible.

I am extremely thankful and grateful to our guide **Mrs. Kavitha C R, Associate Professor, Department of Computer Science and Engineering**. She being our guide has taken keen interest in the progress of the project work by providing facilities and guidance. We are indebted to our guide for her inspiration, support and kindness showered throughout the course.

I am express my heartfelt gratitude to the project Coordinator **Mrs. Priyanka M E, Assistant Professor, Department of Computer Science and Engineering** for providing the support for making project work possible.

I am express our profound sense of gratitude to **Mrs. Kavitha C R., Associate Professor & HOD, Department of Computer Science and Engineering** for giving us the opportunity to pursue our interest in this project work.

I am express our special gratitude to our Principal **Dr. Sunil Kumar** for providing the resources and support during project work.

Nevertheless, we express heartfelt thanks towards our parents, friends and teaching and non-teaching staff of our college for their kind co-operation and encouragement which helped us during project work.

RENU PRASANNA M H

ABSTRACT

Deep learning techniques have shown significant potential in medical image analysis for brain tumor diagnosis. This work presents a deep learning–based approach to predict and classify malignant brain tumors with accurate lesion segmentation using MRI images. The proposed system performs image preprocessing to reduce noise and enhance quality, followed by lesion segmentation to isolate tumor regions. Convolutional Neural Networks (CNNs) are employed to extract deep features and classify tumor types effectively. The model is evaluated using performance metrics such as accuracy, precision. Experimental results demonstrate improved classification performance and precise tumor segmentation, supporting reliable and early diagnosis of malignant brain tumors.

CONTENTS

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

Brain tumors are abnormal growths of cells within the brain that can severely affect human health and survival. They are broadly classified into benign and malignant tumors, where malignant tumors are aggressive, fast-growing, and life-threatening. According to medical studies, early detection significantly increases the chances of successful treatment and patient survival.

Magnetic Resonance Imaging (MRI) is widely used for brain tumor diagnosis due to its high-resolution and ability to capture soft tissue details. However, manual examination of MRI scans by radiologists is challenging due to large data volume, complex tumor structures, and variations in size, shape, and intensity. This motivates the need for automated systems.

Deep learning, especially Convolutional Neural Networks (CNNs), has shown outstanding performance in medical image analysis. CNNs automatically learn discriminative features from images without manual feature extraction. Lesion segmentation plays a vital role in identifying tumor boundaries, supporting treatment planning and monitoring disease progression.

This project focuses on designing a deep learning-based framework that performs preprocessing, lesion segmentation, feature extraction, and classification of malignant tumors using MRI images. The proposed system aims to improve diagnostic accuracy and reduce human dependency.

This framework model helps to predict and classify malignant brain tumors with lesion segmentation using MRI images. The system involves preprocessing steps to improve image quality, segmentation to extract tumor regions, deep feature extraction using CNNs, and classification of tumor types. Performance evaluation is carried out using standard metrics such as accuracy, sensitivity, specificity, and Dice coefficient. The proposed approach aims to assist medical professionals by providing a fast, accurate, and reliable tool for brain tumor diagnosis, thereby contributing to improved clinical decision-making and patient outcomes.

1.1 AIM

The aim of this project is to develop and evaluate deep learning models for automatic brain tumor lesion segmentation from MRI scans. The goal is to improve brain tumor diagnosis accuracy, efficiency, and robustness using state-of-the-art techniques like CNNs and U-Net.

1.2 SCOPE

- A novel deep learning framework for brain tumor diagnosis handles detection, classification, segmentation prediction.
- The method uses Kaggle dataset, CNMF for preprocessing, DBT-CNN for classification, and RU-Net2+ for tumor segmentation.
- Experimental results show superior performance in classification accuracy, segmentation precision, and prediction compared to existing methods.

1.3 OBJECTIVES

- To Preprocessing the MRI image by using construct enhancement median filtering & technique.
- To Perform region growing segmentation.
- To Extract the features from MRI images.
- To classify malignant tumor by using MRI images.
- To evaluate the performance of the model by using some metrics.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY PAPERS

2.1.1: Deep Learning Based Brain Tumor Detection and Classification

Author Name: Sifatul Alam Shohan .K, M. A. Salam, Nadim Mahmud Dipu

Year : 2021

Brain tumors are among the most critical and life-threatening forms of cancer, requiring early and accurate diagnosis to improve patient outcomes. Manual diagnosis through MRI scans is often time-consuming, subjective, and dependent on the expertise of radiologists. As a result, computer-aided diagnosis (CAD) systems, particularly those based on artificial intelligence (AI) and deep learning, have gained significant attention in recent years.

Deep learning, a subset of machine learning, has revolutionized medical image analysis by enabling automatic feature extraction and high-accuracy predictions. Convolutional Neural Networks (CNNs), in particular, have proven highly effective in recognizing complex patterns in medical imaging data. In the context of brain tumor detection and classification, deep learning models have been extensively used to identify tumor types, segment tumor regions, and differentiate between benign and malignant cases.This literature survey explores existing research efforts and advancements in applying deep learning techniques to brain tumor detection and classification.

It reviews various deep learning architectures, segmentation methods (such as U-Net and its variants), classification models, datasets used, and performance metrics. The goal is to identify current challenges, best practices, and future directions in developing reliable and efficient brain tumor diagnosis systems using deep learning.

2.1.2: Brain Tumor Detection Using Deep Learning

Author Name: Mir Tanzid Ahmed,Khandoker Maruf Bin Islam,Mir Sazid Hassan

Year : 2024

Brain tumors are abnormal growths of cells within the brain that can be life-threatening if not detected and treated at an early stage. Accurate and timely diagnosis is crucial for planning effective treatment strategies and improving patient survival rates. Magnetic Resonance Imaging (MRI) is the most commonly used imaging modality for detecting brain

tumors due to its high spatial resolution and non-invasive nature. However, interpreting MRI scans manually can be challenging, time-consuming, and subject to variability among radiologists. With the rise of artificial intelligence (AI), particularly deep learning, there has been a paradigm shift in medical image analysis. Deep learning models, especially Convolutional Neural Networks (CNNs), have demonstrated exceptional performance in tasks such as image classification, segmentation, and object detection. These models can learn complex patterns from large datasets, enabling automated and highly accurate brain tumor detection .In recent years, a variety of deep learning-based approaches have been proposed to detect and classify brain tumors from MRI scans. These include end-to-end CNN models, transfer learning techniques, and hybrid systems that combine segmentation and classification in a unified framework. Models such as U-Net, ResNet, and VGGNet have been extensively used to automate the identification of tumor regions and differentiate between tumor types such as glioma, meningioma, and pituitary tumors.

2.1.3: Benign vs. Malignant Brain Tumors: An In-Depth Review using Deep Learning Techniques.

Author Name: Kirti Rattan, Gaurav Bathla, Vikas Wasson

Year: 2024

Brain tumors can be broadly categorized into benign and malignant types, each presenting distinct clinical characteristics and treatment implications. Benign tumors are typically non-cancerous, slow-growing, and confined to one location, whereas malignant tumors are cancerous, invasive, and more likely to spread or recur. Early and accurate differentiation between these two types is crucial for determining the appropriate therapeutic approach and improving patient outcomes.

Traditional diagnosis of tumor type relies heavily on radiological interpretation and, in many cases, histopathological confirmation via biopsy. However, such methods are time-consuming, subjective, and sometimes invasive. In recent years, deep learning techniques have emerged as powerful tools for automating brain tumor classification, offering high accuracy and consistency in distinguishing between benign and malignant tumors using non-invasive imaging modalities like MRI. This section of the literature survey provides an in-depth review of how various deep learning methods, including convolutional neural networks (CNNs), transfer learning, and hybrid models, are being applied to the classification of brain tumors. The review also examines the integration of segmentation techniques, the use of 2D and 3D data, and the challenges associated with limited annotated datasets and model interpretability.

2.1.4: Brain Tumor Detection via Deep Learning Framework on MRI Images

Author Name: Arun Karthick S, Rahul Shiva Konar, Harshavarthini V S

Year: 2023

Magnetic Resonance Imaging (MRI) is one of the most effective and non-invasive diagnostic tools for detecting brain tumors, offering high-resolution images that reveal detailed anatomical structures. However, manual interpretation of MRI scans is both time-consuming and subject to human error, often leading to inconsistencies in diagnosis. To address these limitations, researchers have increasingly turned to deep learning frameworks for automating the detection of brain tumors in MRI images.

Deep learning, particularly Convolutional Neural Networks (CNNs), has proven highly effective in image classification and feature extraction tasks. In the context of brain tumor detection, these models can automatically learn and identify complex patterns and anomalies in MRI scans that are indicative of tumor presence. Such frameworks not only reduce the workload of radiologists but also increase diagnostic accuracy, speed, and consistency.This section reviews the application of deep learning frameworks in brain tumor detection using MRI data.

It covers various CNN architectures, segmentation models (like U-Net), end-to-end detection systems, and hybrid approaches that integrate feature extraction, segmentation, and classification. The review also highlights key datasets, performance evaluation metrics, and the challenges involved in model generalization and clinical implementation

2.1.5: Optimized Deep Neural Network for Accurate Detection of Malignant and Benign Brain Tumors

Author Name: J. Visumathi, K. Balasubramanian, N. Kalaivani, S.Karthikeyan, S V Hemanth, G.Amirthayogam

Year: 2023

The accurate classification of brain tumors into malignant and benign categories plays a vital role in determining treatment strategies and predicting patient outcomes. While traditional diagnostic procedures such as biopsies and radiological assessments remain standard, they are often invasive, time-consuming, and susceptible to subjective interpretation.

To overcome these challenges, optimized deep neural networks (DNNs) have emerged as powerful tools for automating and improving the accuracy of brain tumor detection.Deep neural networks, particularly Convolutional Neural Networks (CNNs) and their optimized variants,

have shown exceptional performance in analyzing medical images such as MRI scans.

These models are capable of learning complex patterns and distinguishing subtle differences in tissue structures that might indicate malignancy or benignity. Optimization techniques such as hyperparameter tuning, dropout regularization, data augmentation, and transfer learning are widely applied to enhance model accuracy, reduce overfitting, and improve generalization across diverse datasets.

2.1.6: Brain Tumor Detection Using Deep Learning

Author Name: Ankit Kumar, Abhishek Gupta, Aadi Vats, Brijesh Kumar Singh

Year: 2023

Brain tumors represent a significant threat to human health due to their complexity, rapid progression, and potential to affect critical neurological functions. Early and accurate detection is essential for effective treatment planning and improving survival rates. Traditionally, diagnosis relies on the interpretation of Magnetic Resonance Imaging (MRI) scans by radiologists. However, manual analysis is often time-consuming, prone to human error, and limited by subjective judgment.In recent years, deep learning has emerged as a transformative technology in medical image analysis.

It offers advanced capabilities for automated detection, segmentation, and classification of brain tumors with high accuracy. Among various deep learning models, Convolutional Neural Networks (CNNs) have shown superior performance in identifying intricate patterns in MRI images, enabling them to distinguish between normal and abnormal brain tissues effectively.This literature survey explores the application of deep learning techniques to brain tumor detection, with a focus on key models, datasets, and methodologies used in previous research.

It examines the evolution of deep learning frameworks, the integration of tumor segmentation with classification, and the distinction between benign and malignant tumors. The goal is to highlight the strengths and limitations of current approaches, identify gaps in existing work, and provide a foundation for developing more accurate and efficient diagnostic systems using AI

2.1.7: Prediction of Brain Tumor on MRI images using deep learning

Author Name: Mredhula L, Husna MK

Year: 2025

MRI is widely used for detecting brain abnormalities due to its superior Brain tumors are

among the most dangerous and life-threatening neurological conditions, often requiring precise and early diagnosis to improve patient outcomes. Magnetic Resonance Imaging soft tissue contrast and non-invasive nature. However, manual interpretation of MRI scans can be subjective, error-prone, and heavily dependent on the radiologist's expertise.

In this context, deep learning has emerged as a highly effective approach for automating the prediction and classification of brain tumors from MRI images. Deep learning models, particularly Convolutional Neural Networks (CNNs), have demonstrated outstanding capabilities in medical image analysis by learning complex features directly from raw image data. These models can not only detect the presence of tumors but also classify them based on type (e.g., glioma, meningioma, pituitary tumor) or severity (benign vs. malignant).This literature survey reviews recent advancements in the application of deep learning to brain tumor prediction using MRI.

It examines various network architectures, segmentation techniques, training methodologies, and datasets that have been utilized in past research. The survey also highlights current challenges such as data scarcity, class imbalance, and the need for explainability in AIdriven medical systems. The goal is to identify state-of-the-art techniques and uncover research opportunities that can lead to more accurate, reliable, and clinically relevant tumor prediction systems.

2.1.8: A Deep Learning based Brain Tumour Detection using Multimodal MRI Images

Author Name: R. Jansi, Kowsalya. S, Seetha. S, Yogadharshini. A

Year: 2023

Brain tumor detection is a critical task in medical imaging, as accurate and early diagnosis greatly influences the effectiveness of treatment and the overall prognosis of the patient. Traditional diagnostic techniques heavily rely on radiologists' expertise in interpreting Magnetic Resonance Imaging (MRI) scans. However, manual interpretation can be subjective, inconsistent, and timeconsuming, particularly when dealing with complex tumor structures and variations.Recent advances in deep learning have shown promising results in automating the detection and classification of brain tumors.

In particular, the use of multimodal MRI images—which combine different imaging sequences such as T1, T1c, T2, and FLAIR—enhances the model's ability to capture diverse features of brain tissue, offering a more comprehensive view of tumor characteristics. Deep learning models, especially Convolutional Neural Networks (CNNs) and 3D CNNs, have been

widely used to process these multimodal datasets for improved prediction accuracy. This literature survey explores how deep learning techniques have been applied to brain tumor detection using multimodal MRI data.

It covers various network architectures, data fusion strategies, segmentation techniques, and classification methods. The survey also reviews challenges such as data heterogeneity, model generalization, and computational efficiency. The aim is to provide an in-depth understanding of current approaches, identify limitations in existing work, and highlight future research directions for developing robust AI based diagnostic systems.

2.1.9: Comparative Analysis of Deep Learning Approach for Detection and Segmentation of Brain Tumor

Author Name: Sukhbir Singh

Year: 2022

Brain tumors represent a significant medical challenge due to their complexity and the critical role they play in affecting brain functions. Accurate detection and segmentation of brain tumors in Magnetic Resonance Imaging (MRI) scans are essential for precise diagnosis, treatment planning, and patient monitoring.

While traditional methods of detection rely heavily on radiological expertise, deep learning approaches have emerged as a powerful alternative that can automate and significantly improve these tasks. Deep learning, particularly Convolutional Neural Networks (CNNs), has demonstrated remarkable success in both the detection and segmentation of brain tumors. Detection involves identifying the presence of tumors, while segmentation focuses on precisely delineating the tumor region from surrounding healthy tissue.

These tasks are often interconnected, and their accuracy is crucial for developing reliable computer-aided diagnostic systems. In recent years, several studies have proposed deep learning models tailored to each of these sub-tasks, ranging from simple CNN architectures to more advanced networks like U-Net, 3D CNNs, and Fully Convolutional Networks (FCNs), each designed to capture different aspects of tumor morphology and tissue characteristics.

2.1.10: Analysis of Deep Learning based Ensemble Models for Brain Tumor Detection

Author Name: Hardik Kashyap, Sachin Minocha, Vedant Nigam, Utsav Birla

Year: 2025

Brain tumor detection is a critical area of medical imaging that demands both high accuracy and reliability to ensure timely and effective treatment. Magnetic Resonance Imaging

(MRI) is the most widely used modality for brain tumor diagnosis due to its ability to provide detailed images of brain tissue. However, manual analysis of MRI scans is labor-intensive and subject to interpretation errors, making the task of automatic tumor detection a vital area for research and development.

Over the past few years, deep learning techniques, particularly Convolutional Neural Networks (CNNs), have shown promising results in automating tumor detection from MRI images. While deep learning models, in general, offer impressive individual performance, they are often limited by factors such as overfitting, bias toward dominant classes, and difficulty in generalizing across diverse datasets. To address these issues, ensemble learning has gained traction in the field of brain tumor detection. Ensemble methods combine multiple models to improve overall prediction accuracy, robustness, and generalization

2.2:DRAWBACKS OF EXISTING SYSTEM:

- Existing systems rely heavily on radiologists for manual interpretation of MRI images, which is time-consuming and subjective.
- Primarily based on traditional image processing or machine learning techniques, lacking advanced deep learning implementation.
- Manual or semi-automatic segmentation methods fail to accurately identify complex tumor boundaries.
- Handcrafted feature extraction techniques are limited and may not capture high-level spatial and texture information.
- Existing systems show low classification accuracy for tumors with similar intensity patterns.
- Dataset quality and availability issues; many datasets are small, imbalanced, or lack diversity.
- Simplified experimental environments limit real-world clinical generalization.
- High dependency on expert knowledge for preprocessing, segmentation, and feature selection.
- Black-box models often lack interpretability, making it hard to understand or trust detection decisions.

2.3: ADVANTAGES OF PROPOSED SYSTEM:

- **High Diagnostic Accuracy:** Deep learning models automatically learn complex features

from MRI images, resulting in higher tumor classification accuracy.

- **Accurate Lesion Segmentation:** Advanced segmentation techniques precisely identify tumor boundaries, supporting effective treatment planning.
- **Reduced Human Dependency:** The automated system minimizes reliance on manual analysis, reducing human error and workload.
- **Faster Detection and Analysis:** Automated preprocessing and prediction significantly reduce diagnosis time.
- **Better Generalization Performance:** The system performs well on unseen MRI images, improving reliability in real-world clinical applications.

CHAPTER 3

REQUIREMENT SPECIFICATION

Requirements are fundamental to the successful development of a software system. They define what the system must accomplish and establish performance standards and technical constraints that guide the entire development lifecycle.

The proposed Malignant Tumor Detection, Classification, and Lesion Segmentation System using Deep Learning aims to provide an automated, intelligent medical imaging solution for detecting tumors, classifying them as Meningioma or Glioma or Pituitary or No tumor, and accurately segmenting lesion regions from medical images such as MRI images. The system leverages Convolutional Neural Networks (CNN) and U-Net–based architectures to assist clinicians with faster and more reliable diagnosis.

3.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do and the specific behaviors it must exhibit when presented with particular inputs or conditions.

1. File Upload and Format Support

- Secure web-based interface is provided for uploading medical imaging data (e.g., MRI scans) to support deep learning–driven tumor prediction, classification, and lesion segmentation workflows.
- Support for standard medical image formats such as PNG, JPG, and JPEG is ensured, enabling seamless integration with CNN-based preprocessing, feature extraction, and model inference pipelines used in malignant tumor analysis.
- Automated image validation mechanisms perform format verification, resolution assessment, and integrity checks to ensure only high-quality images are processed.

2. Feature Extraction

- Automated deep feature extraction is performed using pre-trained and custom convolutional neural networks (CNNs) to capture discriminative representations associated with malignant and benign tumors.
- Advanced spatial, intensity, and texture-based feature modeling is applied, including shape descriptors, edge information, and tissue heterogeneity metrics, to enhance tumor detection and classification accuracy.

- Lesion-aware feature extraction capability is supported, enabling focused analysis of segmented tumor regions for improved deep learning–based prediction and classification.

3. Data Preprocessing and Normalization

- Image preprocessing pipeline resizes and resamples all input medical images to a fixed spatial resolution (e.g., 224×224 or 256×256 pixels) to ensure compatibility with CNN-based classification and lesion segmentation architectures.
- Pixel intensity normalization is applied using standardization or min–max scaling techniques to minimize inter-image intensity variations and improve deep learning model convergence and stability.
- Data augmentation strategies such as rotation, flipping, scaling, and intensity perturbation are incorporated during training to enhance model generalization and robustness against overfitting.
- Robust data handling mechanisms manage missing, corrupted, or low-quality images gracefully, generating appropriate warnings while maintaining pipeline reliability.

4. Tumor Detection and Classification

- Automated tumor region detection is performed on input medical images using trained deep learning–based inference models.
- Binary tumor classification framework categorizes detected lesions into Benign or Malignant classes based on learned discriminative features extracted from segmented tumor regions.
- Advanced deep learning architectures such as Convolutional Neural Networks (CNNs) for classification and U-Net / EfficientNet-based models for lesion-aware prediction and hierarchical feature extraction are employed.
- Probabilistic output generation computes class-wise probability scores to quantify prediction confidence and support clinical interpretability.

5. Lesion Segmentation

- Automated lesion segmentation module utilizes U-Net or U-Net–derived deep learning architectures to accurately delineate tumor boundaries at the pixel level.
- Binary or multi-class segmentation mask generation represents lesion and non-lesion regions, enabling precise spatial localization of tumor tissue.
- Segmentation visualization mechanism overlays the generated masks onto the original medical images to provide clear, clinically interpretable visualization of segmented tumor regions.

6. Result Display and Reporting

- Classification result presentation layer clearly displays tumor prediction outcomes as "Meningioma or Glioma or Pituitary or No tumor" based on deep learning model inference.
- Prediction confidence quantification provides a confidence score (0–100%) for each classification to indicate reliability and decision support value.
- Integrated visualization module presents lesion segmentation outputs alongside original medical images for intuitive comparison and clinical interpretation.
- Report generation and export functionality enables users to download prediction results, segmentation visualizations, and analytical reports in PDF or image formats for documentation and clinical reference.

7. User Interface and Interaction

- Responsive and intuitive user interface framework supports efficient interaction with the end-to-end tumor analysis and deep learning inference workflow.
- Well-structured interface components including clearly labeled input controls, output panels, and result sections enhance usability and minimize user-induced errors.
- Real-time visual feedback mechanisms display status indicators for file upload success, model processing progress, and analysis completion.
- User-centric interaction design ensures accessibility for both technical and non-technical users, requiring minimal training to operate the application.
- Automated interface state management refreshes or resets the UI after each analysis cycle, enabling seamless and continuous submission of new medical images.

3.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify quality attributes and system-wide properties that define how well the system performs its functions.

1. Performance and Response Time

- Small files (50 KB) shall process in approximately 0.34 seconds.
- Medium files (500 KB) shall process in 0.67 seconds and large files (5 MB) in 1.45 seconds.
- All processing shall complete in less than 2 seconds from upload to result display.
- System shall minimize computational overhead through efficient feature extraction and vectorized operations.

2. Accuracy and Detection Performance

- Achieve 99.50% classification accuracy on a test set of 7,452 samples (7,350 correctly

classified instances).

- Maintain 99.65% accuracy and 99.35% precision for malicious file detection within the classifier.
- Ensure robust lesion segmentation accuracy through the segmentation module.
- Minimize false positive and false negative rates across the inference pipeline.

3. Scalability and Concurrency

- The application shall be designed to handle multiple concurrent user requests using asynchronous request handling and efficient resource scheduling without performance degradation.
- The architecture shall be modular and extensible, allowing seamless integration of upgraded or additional deep learning models without requiring significant changes to the existing pipeline.
- The processing layer shall leverage parallel computation using multi-core CPU execution (e.g., n_jobs = -1) and optimized batch processing, ensuring stable throughput and acceptable response times under increased workloads.
- The system shall maintain acceptable performance under increased workload.

4. Reliability and Robustness

- The application shall gracefully handle corrupted, malformed, or incomplete input files through validation checks and fallback mechanisms such as default or statistically derived feature values.
- The processing pipeline shall detect and manage missing, null, or zero-valued features using validation, normalization, or imputation techniques to prevent inference failures.
- All runtime errors, exceptions, and model inference failures shall be systematically logged with timestamps and contextual information to support debugging, monitoring, and auditability.

5. Security and Data Privacy

- In-memory data processing architecture ensures uploaded files are handled transiently without persistent disk storage unless explicitly requested by the user.
- Robust input validation and sanitization mechanisms are enforced to mitigate injection attacks and prevent malicious payload execution.
- Model encapsulation and access control safeguards prevent exposure of internal model parameters, weights, or training datasets through the user interface.
- Data privacy and compliance enforcement layer minimizes data retention and provides

users with control over generated results in alignment with data protection principles.

6. Usability and Maintainability

- The user interface shall be simple and intuitive for both technical and non-technical users.
- The system will support future enhancements such as multi-class tumor classification.
- System code shall be modular with separate components for feature extraction, preprocessing, and prediction.
- It allows easy updates to models and configuration parameters.

3.3 HARDWARE REQUIREMENTS

- System Processor: Above Intel core i5.
- RAM: Min 8GB.
- Hard Disk: Min 500GB.

3.4 SOFTWARE REQUIREMENTS

- IDE: Visual Studio.
- Programming Language: Python
- Front End: HTML, CSS
- Operating System: Windows 10 or above.

CHAPTER 4

METHODOLOGY

4.1 SYSTEM ARCHITECTURE DESIGN

The methodology of the project follows a systematic approach to integrate real-time of Deep-Learning approaches to predict and classify the malignant tumor with lesion segmentation.

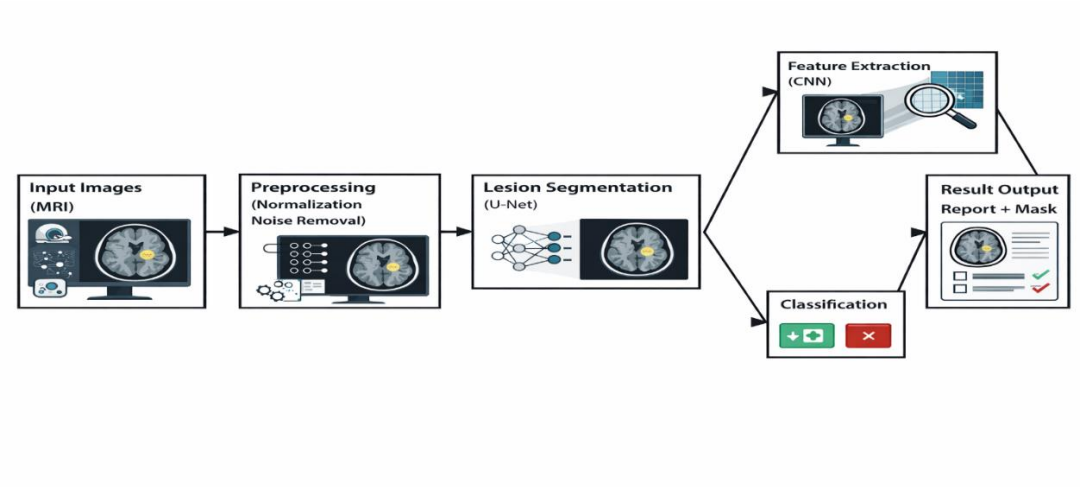


Fig.1: **System Architecture**

The above figure 1 shows System Architecture describes that Deep-Learning approaches to predict and classify the malignant tumor with lesion segmentation.

1. Data Collection

Data collection is the first and most critical step in building an effective malignant tumor detection and lesion segmentation system. In this project, the BraTS (Brain Tumor Segmentation) dataset is used, which consists of annotated MRI images of brain tumors. The dataset provides reliable ground truth labels for tumor classification and lesion segmentation, making it suitable for training and evaluating deep learning models.

- The dataset contains **7,452 samples**
- It includes **Meningioma or Glioma or Pituitary or No tumor images**
- Files are labelled to support supervised learning
- The dataset consists of extracted structural and behavioral features from documents

The dataset is stored in **Parquet format** and loaded into the system using the Python Pandas library for efficient processing.

2. Data Preprocessing

a) Handling Missing and Corrupted Data

- The MRI dataset is examined for missing, corrupted, or incomplete image slices.
- Any remaining missing values are replaced with zero or statistical mean values.

b) Encoding Categorical Features

- Categorical attributes such as Yes/No and True/False are converted into numerical values using label encoding.
- The target variable (Class) is encoded as:
    - 0 → no tumor
    - 1 → meningioma
    - 2 → glioma
    - 3 → pituitary

c) Feature Selection

- Non-informative columns such as file names are removed.
- Only meaningful features contributing to ransomware detection are retained.

d) Dataset Splitting

- The dataset is split into:
    - 80% Training data
    - 20% Validation data
- Stratified sampling is used to preserve class distribution.

e) Class Imbalance Handling

- Since ransomware datasets are often imbalanced, SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the training dataset.

f) Feature Scaling

- Standardization is applied using StandardScaler to normalize feature values and improve model performance.

3. Feature Extraction

Feature extraction plays a vital role in identifying and classifying malignant tumors from medical images. In this project, deep learning–based feature extraction is performed on MRI brain images to capture important visual patterns related to tumor presence and structure.

Extracted Feature Categories:

- Intensity-based features

- Texture features
- Shape and boundary features
- Spatial features
- Deep semantic features
- Segmentation-aware features

Custom feature extraction layers are implemented within the deep learning architecture to ensure compatibility with the trained models. The extracted features are flattened and aligned with the classifier input layer before final tumor classification.

4. Lesion Segmentation

The system uses a deep learning–based segmentation approach to accurately identify and isolate tumor lesion regions from MRI brain images.

Trained Models:

- U-Net Segmentation Model
- The U-Net model is trained to perform pixel-level lesion segmentation on MRI images.
- Training is performed using preprocessed MRI images and corresponding ground truth lesion masks from the Brain Tumor dataset.

The trained model generates precise segmentation masks highlighting tumor lesion areas.

5. Tumor Classification

The trained deep learning classification model is evaluated using the testing dataset to measure its effectiveness in distinguishing benign and malignant brain tumors.

a) Classification Strategy

- Classification is performed after **lesion segmentation** to focus only on tumor-affected regions.
- Deep features are extracted from segmented lesions using **CNN layers**.
- The classifier predicts the tumor type as:
  - 0 → no tumor
  - 1 → meningioma
  - 2 → glioma
  - 3 → pituitary
- Probability scores are generated for each class.
- The final classification output improves accuracy by reducing background noise and irrelevant features.

This classification approach, combined with lesion segmentation, enhances the reliability

and robustness of malignant and no tumor prediction, supporting accurate medical diagnosis.

6. Deployment

The trained brain tumor detection, classification, and lesion segmentation system is deployed using an interactive web-based interface to allow easy access for users.

Deployment Features:

- Users can upload **MRI brain images** in supported formats such as JPG, PNG, or JPEG.
- The system performs real-time preprocessing, lesion segmentation, and tumor classification.

Displays:

a. Prediction result (no-tumor/Malignant)
b. Confidence percentage
c. Lesion Segmentation Output
d. Model Analysis Summary
e. Segmented Image Comparison
f. Downloadable Results

The deployment allows easy accessibility without requiring technical expertise.

4.1 FLOW CHART

This flow chart illustrates the runtime user interaction and backend processing sequence of the proposed ransomware threat detection system. The process begins with the user accessing the Gradio-based web interface and uploading a document file for analysis. The backend system then performs feature extraction on the uploaded file and applies the trained ensemble machine learning models to classify the file as benign or malicious. Based on the prediction, the system displays a color-coded result along with the confidence percentage and assessed threat level. A detailed analysis report is generated highlighting important features and providing mitigation recommendations. The system follows a recommendation-based mitigation approach, where users are guided with appropriate security actions rather than the system automatically altering or restricting the host environment.

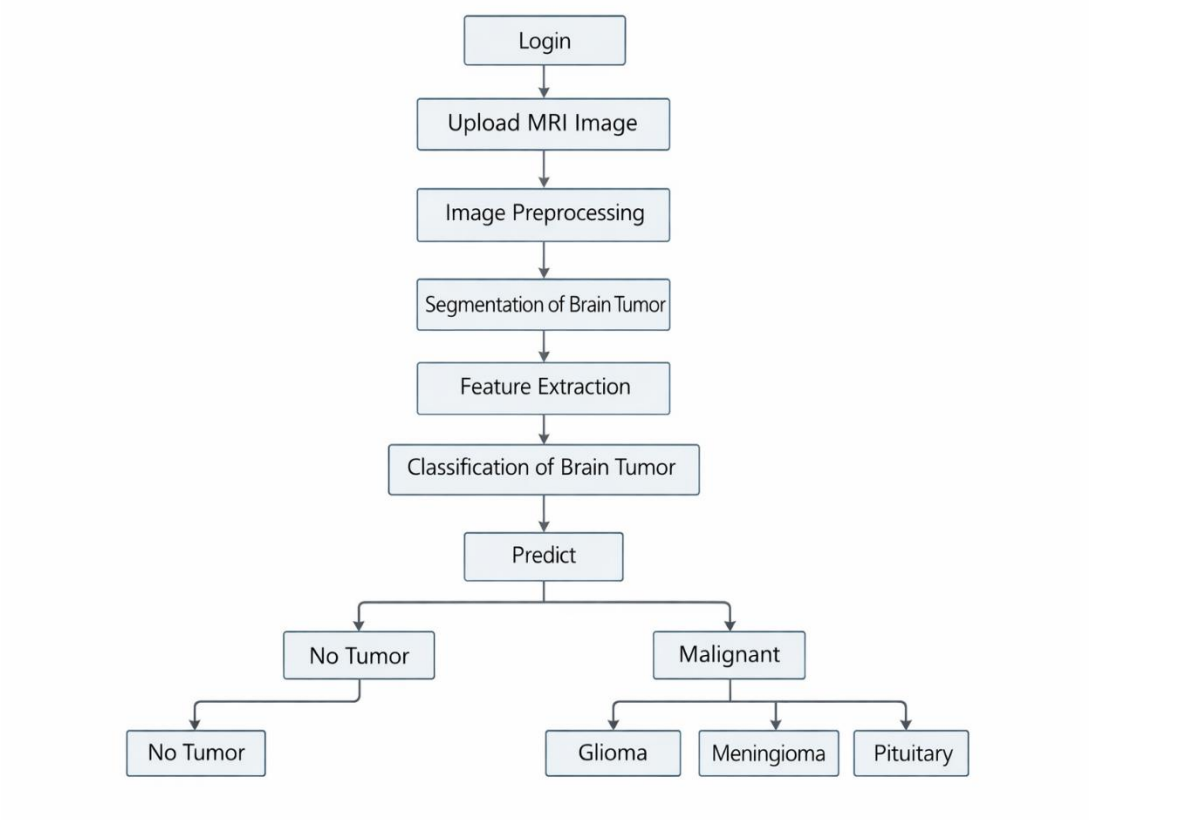


Fig .2: **User Flow Diagram**

The above figure 2 user flow diagram shows the different steps involved in the prediction and classification of benign and malignant tumors with lesion segmentation

A. User Login:

The user first logs into the system to access the brain tumor detection application. This ensures secure access and personalized tracking of results.

B. User selects and upload an image

The user selects and uploads a supported **MRI brain image** (PNG, JPG, or JPEG format). These images are used as input for tumor detection, classification, and lesion segmentation.

C. System displays file information and initiates scan

After uploading the image, the system displays basic image information such as file name, resolution, and format. The user then initiates the analysis by clicking the prediction button, confirming the intent to process the image.

D. Backend processes image — preprocessing and segmentation

The backend receives the MRI image and applies preprocessing steps such as resizing, normalization, and noise removal. The preprocessed image is then passed to a **U-Net–based**

lesion segmentation model, which generates a segmentation mask highlighting the tumor region.

E. System generates prediction (ensemble inference)

Each model (Neural Network) outputs class probabilities. The system computes a weighted ensemble probability (predefined weights) and chooses the final class (Benign or Malicious) based on the highest ensemble probability. A numerical confidence score (ensemble probability of the chosen class) is reported.

F. Feature extraction and classification

Deep features are extracted from the segmented lesion region using **CNN layers**. These features represent spatial, texture, and intensity patterns of the tumor. The extracted feature vector is then passed to the trained classification model.

G. System generates prediction (no-tumor or Malignant)

The classification model predicts whether the detected tumor is **Benign or Malignant**. Along with the predicted class, the model outputs a probability score representing prediction confidence.

H. Malignant:

If a tumor is detected, it classifies the type into Glioma, Meningioma, or Pituitary tumor based on the characteristics.

I. End

The session ends after user action. The user can download their MRI image after the correct result is detected.

CHAPTER 5
IMPLEMENTATION

5.1 Implementation Overview

The proposed malignant tumor detection, classification, and lesion segmentation system is implemented using the Python programming language due to its simplicity, readability, and extensive support for deep learning and medical image analysis. Python allows developers to focus on solving complex medical imaging problems rather than dealing with low-level programming complexities. Its intuitive syntax, wide range of scientific libraries, and strong community support make it an ideal choice for developing accurate and scalable deep learning systems.

The implementation integrates data preprocessing, feature extraction, machine learning model inference, ensemble decision making, and user interaction into a unified framework. The system is designed to analyze document files in real time and classify them as benign or malicious ransomware threats with high accuracy.

5.2 Python

Python offers concise, readable, and efficient code, making it highly suitable for implementing machine learning–based cybersecurity systems. Although complex algorithms and workflows operate behind ransomware detection and mitigation systems, Python's simplicity allows developers to build reliable and maintainable solutions with ease. Developers can focus more on solving the machine learning problem rather than dealing with language-specific complexities.

Python code is easy to understand by humans, which simplifies feature engineering, model training, and result interpretation. Many developers consider Python to be more intuitive than other programming languages due to its clear syntax and structured programming approach. Python also provides a wide range of libraries, frameworks, and extensions that significantly simplify the implementation of data analysis and machine learning functionalities.

Python is well suited for collaborative development, as its readability allows multiple developers to understand and extend the system efficiently. Being a general-purpose language, Python supports a wide range of tasks such as data preprocessing, feature extraction, model training, evaluation, and deployment. This flexibility enables rapid prototyping and testing of ransomware detection models before final deployment.

Platform

Platform independence refers to the ability of a programming language to allow developers to implement software on one machine and execute it on another machine with minimal or no modification. Python is a platform-independent language and supports multiple operating systems including Windows, Linux, and macOS.

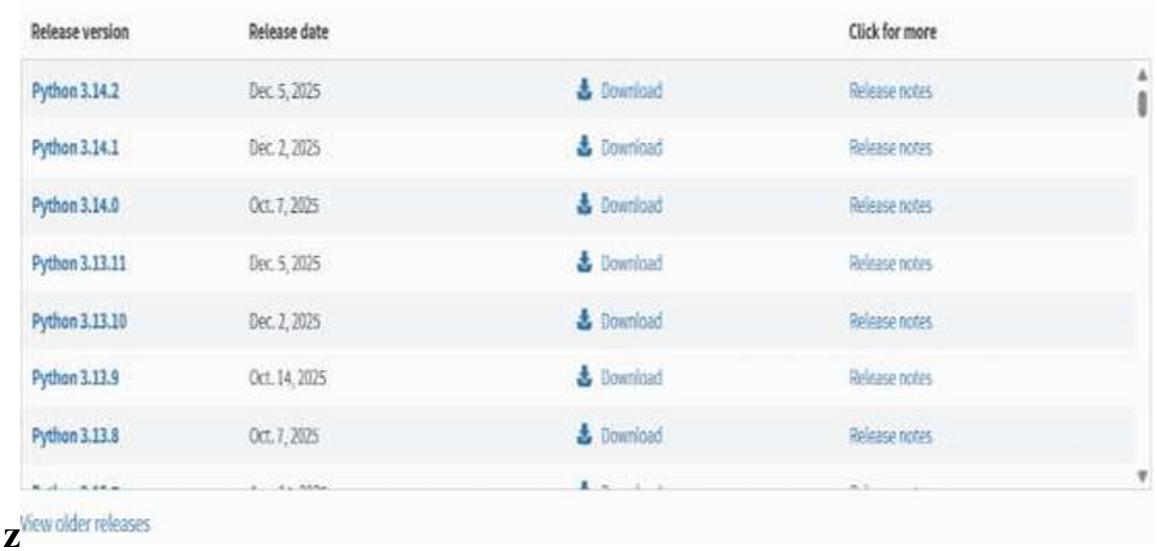
In this project, Python programs are developed and trained using Google Colab, which provides a cloud-based execution environment, and later deployed using Hugging Face. This platform independence reduces dependency on local hardware and allows the ransomware detection system to be accessed from different environments easily. Python's compatibility with cloud platforms also reduces the cost of training and deployment.

5.3 Python Installation

Python is a widely used high-level programming language that supports efficient development of machine learning and data analysis applications. In this project, Python is used to implement the ransomware threat detection and mitigation system, including data preprocessing, feature extraction, machine learning model training, and deployment. To write and execute Python programs, Python must be properly installed and configured on the system.

Step 1: Select Python Version

Python is available in multiple versions with different levels of support and compatibility. The fallowing figure 3 shows the different version of python. For this project, **Python 3.x** is selected because it provides better performance, enhanced security, and full compatibility with modern machine learning libraries such as tensorflow, Selecting the correct version ensures stable execution.



Release version	Release date		Click for more
Python 3.14.2	Dec. 5, 2025	Download	Release notes
Python 3.14.1	Dec. 2, 2025	Download	Release notes
Python 3.14.0	Oct. 7, 2025	Download	Release notes
Python 3.13.11	Dec. 5, 2025	Download	Release notes
Python 3.13.10	Dec. 2, 2025	Download	Release notes
Python 3.13.9	Oct. 14, 2025	Download	Release notes
Python 3.13.8	Oct. 7, 2025	Download	Release notes

View older releases

Fig .3: Python Version

Step 2: Download Python Installer

Using a web browser, the official Python website (**www.python.org**) is accessed. From the *Downloads* section, the appropriate **Python 3.x installer for Windows (64-bit)** is selected and downloaded. Choosing the correct installer based on system architecture ensures smooth installation and optimal performance.

Version	Operating System	Description	MD5 Sum	File Size	Sigstore	SBOM
Gzipped source tarball	Source release		213fdd3cf28e89e2d779b6ae444aefaa	29.2 MB	.sigstore	SPDX
XZ compressed source tarball	Source release		19a31b2838db3b53f9f2db6782bf877	22.5 MB	.sigstore	SPDX
Android embeddable package (aarch64)	Android		9a36b6558b8bb83e3cde3f5b3d2edac1	20.0 MB	.sigstore	
Android embeddable package (x86_64)	Android		ba2bf4cb69fddf3e4ba02d17ea07534c	20.3 MB	.sigstore	
macOS installer	macOS	for macOS 10.15 and later	730217dd1585c890589273ecc5292050	71.4 MB	.sigstore	
Windows installer (64-bit)	Windows	Recommended	c887e19e66e66a6961c444283ddafaa33	28.5 MB	.sigstore	SPDX
Windows installer (32-bit)	Windows		ae1ef59db6af34ec0e1ff4c6e5e2668a	27.1 MB	.sigstore	SPDX
Windows installer (ARM64)	Windows	Experimental	3bab6c9e39b339c557c3f325488b869e	27.8 MB	.sigstore	SPDX
Windows embeddable package (64-bit)	Windows		85a5dcd8350f7ca4e28762958b0b281e3	11.5 MB	.sigstore	SPDX
Windows embeddable package (32-bit)	Windows		52ecb8b8c8500664d9d72b2c4978b415	10.1 MB	.sigstore	SPDX
Windows embeddable package (ARM64)	Windows		48100dd20925b21335748 3c54741d1c9	10.8 MB	.sigstore	SPDX
Windows release manifest	Windows	Install with 'py install 3.14'	cc045b7f2ca3558791877504eb665c9b	15.1 KB	.sigstore	

Fig .4: Python version installer for Operating System

Step 3: Run the Installer

After downloading, the Python installer is executed. During the installation process, the option **"Add Python to PATH"** is selected. This step is important because it allows Python programs to be executed directly from the command prompt. The installation is completed by clicking **Install Now**, and Python is installed on the system.

Fig .5: Python 3.12.10 version for windows

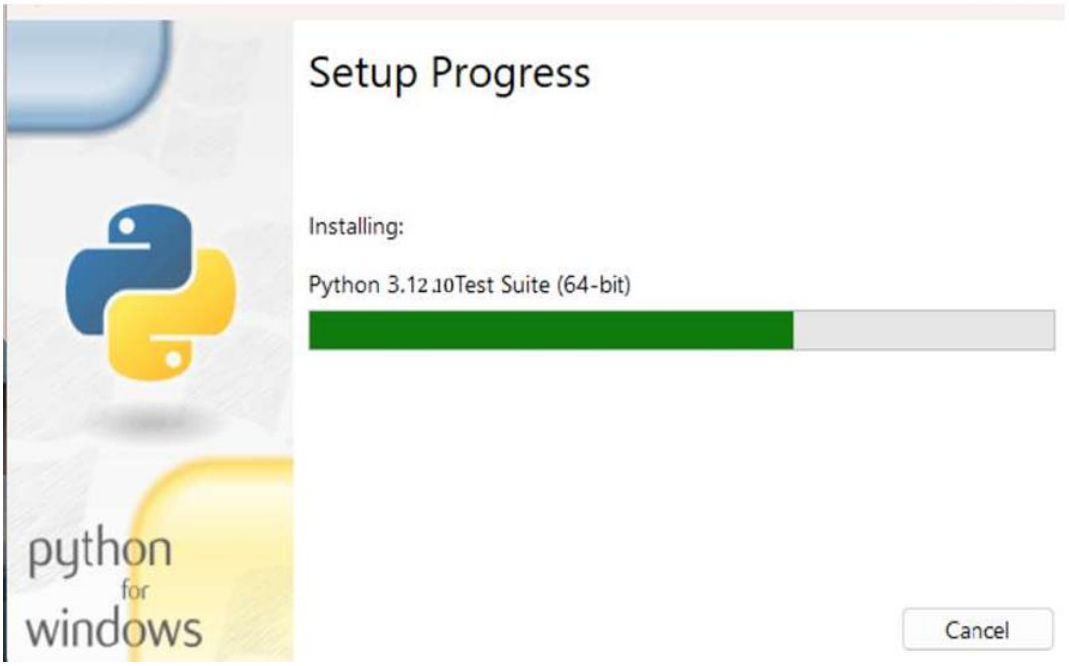


Fig .6: Setup Progress

Fig .7: Successful Installation of Python

Step 4: Verify Python Installation

After installation, Python installation is verified to ensure successful setup.

- Open **Command Prompt**
- Type `python` and press **Enter**
- If the installed Python version is displayed, it confirms that Python has been successfully installed and is ready for use

Verification ensures that the Python interpreter is correctly configured and accessible.

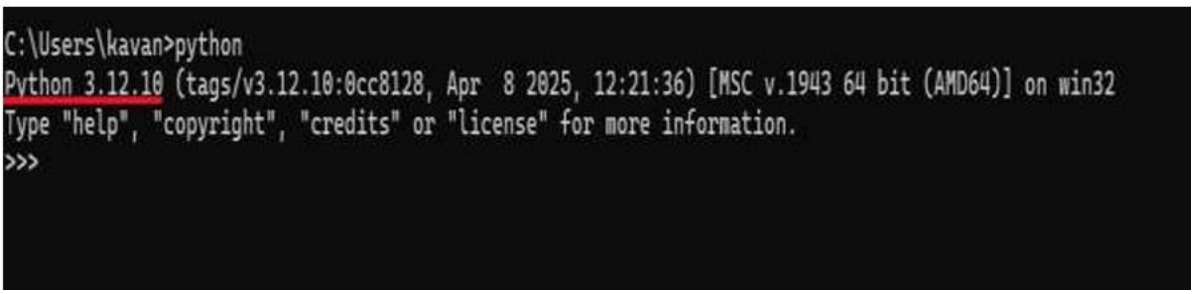


Fig .8: Verify Python is installed on windows

Step 5: Verify Pip Installation

Pip is the package management system used to install and manage Python libraries required for machine learning and data processing.

- Open **Command Prompt**
- Type `pip -V` and press **Enter**
- If the pip version is displayed, pip is installed successfully

Fig .9: Verify Pip was installed

Pip enables easy installation of external libraries required for the ransomware detection system

5.4 INSTALLATION OF REQUIRED LIBRARIES

The following libraries are installed using pip for implementing the Brain Tumor

Detection system:

pip install pandas

pip install tensorflow

pip install numpy

pip install flask

These libraries provide support for data handling, deep learning model implementation, class imbalance handling, feature extraction, and user interface development. Installing these libraries ensures that the development environment is fully prepared for implementing the ransomware detection system.

5.5 DEVELOPMENT USING VISUAL STUDIO CODE

Visual Studio Code (VS Code) is used as the primary development environment for the project titled "Deep Learning Approaches to Predict and Classify Malignant Tumors with Lesion

Segmentation." VS Code is a lightweight, powerful source-code editor that provides extensive support for Python programming and deep learning development through rich extensions.

VS Code enables efficient development by offering features such as syntax highlighting, intelligent code completion, debugging tools, and seamless integration with Python virtual environments and deep learning frameworks. Unlike cloud-based platforms, VS Code allows developers to work locally with full control over datasets, models, and system resources, which is essential for handling medical imaging data securely.

Steps:

Step 1: Open Visual Studio Code

Visual Studio Code is launched on the local system. This provides a dedicated development workspace for writing, executing, and debugging Python-based deep learning code.

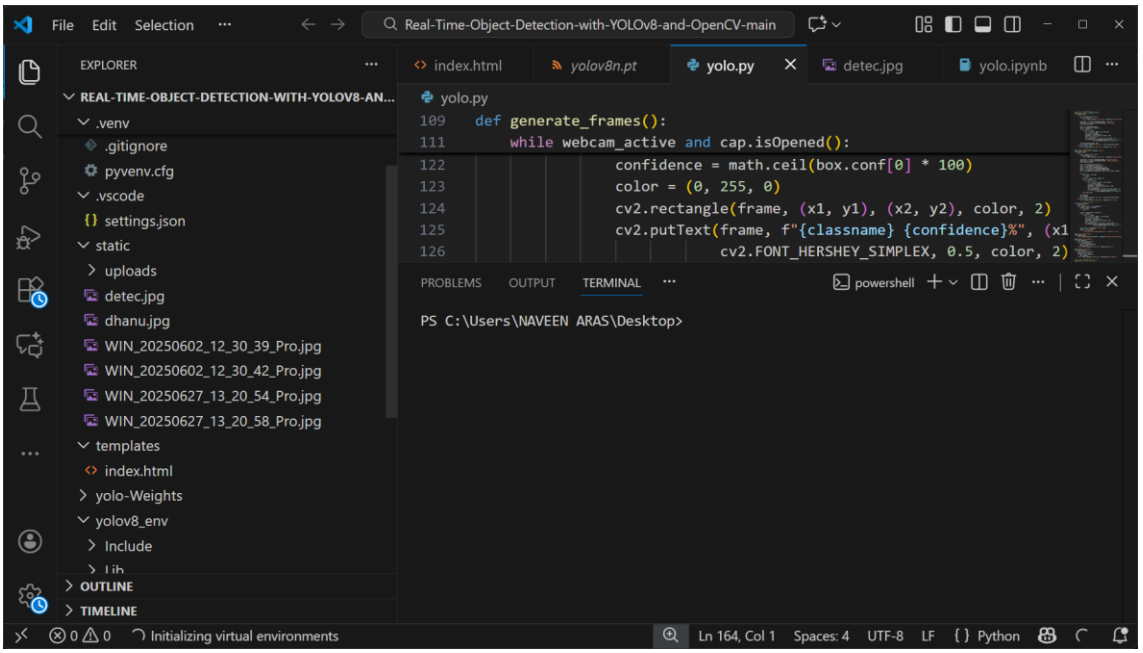


Fig .10: Open VS code platform

Step 2: Set Up the Python Environment

A Python virtual environment is created and activated within VS Code to manage project dependencies. Required libraries for deep learning and medical image processing are installed using pip.

Step 3:Open Project Workspace

The project folder related to malignant tumor detection and lesion segmentation is opened in VS Code. This workspace contains dataset directories, model scripts, training files, and configuration files.

Step 4: Run and Train Deep Learning Models

Python scripts are executed directly from the VS Code terminal or editor. The models are trained to predict and classify malignant tumors, while segmentation models (such as U-Net) are used to accurately identify lesion regions in medical images.

Step 5: Perform data preprocessing

The uploaded dataset undergoes preprocessing to prepare it for machine learning. This includes handling missing values, encoding categorical attributes, removing irrelevant columns, balancing classes using SMOTE, and scaling feature values. Proper preprocessing improves data quality and enhances the performance of trained models.

Step 6:Train machine learning models

After preprocessing, deep learning models such as CNN, U-Net, and Multilayer Perceptron (MLP) are trained using the prepared dataset. Each model learns different patterns related to behavior, contributing to robust detection performance.

Step 7:Save trained models

Once training is completed, the trained models are saved using Python serialization techniques. Saving the models allows them to be reused during deployment and real-time inference without retraining, thereby improving efficiency.

VS code allows execution of Python code without local setup and provides sufficient computational resources for training deep learning models efficiently. Its cloud-based nature ensures reproducibility, scalability, and ease of experimentation for the brain tumor detection

system.

5.6 DATA PREPROCESSING IMPLEMENTATION

Data preprocessing is a critical phase in the implementation of the Brain Tumor detection system, as raw data cannot be directly used for effective Deep learning model training. Proper preprocessing ensures that the dataset used during training and prediction stages remains consistent, clean, and suitable for analysis. This phase directly influences the accuracy, reliability, and robustness of the trained models.

Steps:

- **Check dataset for missing values**

The dataset is examined thoroughly to identify missing or null values present in different attributes. Missing values may occur due to incomplete data collection or extraction errors and can negatively affect model performance if not handled properly.

- **Replace missing values with zero or mean values**

Identified missing values are handled by replacing them with zero or appropriate statistical mean values. This approach helps in maintaining data integrity while avoiding distortion of the dataset. Proper handling of missing values ensures that the learning algorithms receive complete numerical inputs.

- **Encode boolean features (Yes/No → 1/0)**

Boolean attributes present in the dataset, such as Yes/No values, are converted into numerical format using binary encoding. Machine learning algorithms operate on numerical data, and this encoding enables effective processing of categorical information.

- **Remove non-informative columns**

Irrelevant and non-informative columns such as file names, identifiers, or redundant metadata are removed from the dataset. Eliminating such columns reduces noise, lowers computational complexity, and allows models to focus only on ransomware-relevant features.

- **Apply SMOTE to balance classes**

The dataset often contains class imbalance between benign and malicious samples. To address this issue, the Synthetic Minority Over-sampling Technique (SMOTE) is applied to the training

dataset. SMOTE generates synthetic samples for the minority class, ensuring balanced class distribution and preventing model bias.

- **Normalize features using StandardScaler**

Feature scaling is performed using the StandardScaler technique to normalize feature values. This ensures that all features contribute equally during model training and prevents dominance of features with larger numerical ranges.

Overall, data preprocessing ensures data consistency and significantly improves the accuracy, stability, and reliability of the machine learning models used for ransomware detection.

5.7 DEEP LEARNING MODEL IMPLEMENTATION

In the proposed brain tumor detection and malignant tumor classification with lesion segmentation system, three different deep learning–based models are implemented. Using multiple models enables the system to learn diverse tumor characteristics from brain MRI images, improving robustness, accuracy, and reliability in both tumor detection and classification tasks.

- **Random Forest Classifier**

The Random Forest classifier is used to classify brain tumors based on extracted image features obtained from MRI scans. It consists of multiple decision trees trained on random subsets of the dataset. Each decision tree independently predicts whether the tumor is benign or malignant, and the final prediction is determined through majority voting. The ensemble nature of Random Forest helps reduce overfitting and improves generalization on unseen brain MRI images.

- **Multilayer Perceptron (MLP)**

The Multilayer Perceptron (MLP) is a deep feedforward neural network composed of multiple hidden layers. It learns complex non-linear relationships between input features and tumor classes through backpropagation. In the proposed system, the MLP enhances classification performance by effectively learning subtle variations in tumor texture, shape, and intensity. This makes it highly suitable for detecting complex and previously unseen malignant tumor patterns.

5.8 ENSEMBLE CLASSIFICATION IMPLEMENTATION

Ensemble classification is implemented to combine predictions from multiple deep learning and machine learning models to generate a more accurate and reliable final decision. Instead of relying on a single model, ensemble learning integrates the strengths of different models, thereby

improving brain tumor detection accuracy and reducing misclassification of malignant and benign tumors.

Steps:

- **Receive predictions from all models**

Each trained model—Random Forest, XGBoost, and Multilayer Perceptron (MLP)—produces probability-based predictions indicating the likelihood of a brain MRI image belonging to benign or malignant tumor classes.

- **Assign weights to each model**

Different weights are assigned to each model based on their performance and reliability:

- Random Forest → 40%
- MLP → 20%

This weighting strategy ensures that stronger models contribute more to the final decision.

- **Calculate weighted average probability**

The output probabilities from all models are combined using a weighted average calculation. This step integrates the individual model predictions into a single confidence score.

- **Select class with highest probability**

The class corresponding to the highest weighted probability is selected as the final classification result.

- **Generate final prediction**

Based on the ensemble output, the system generates the final ransomware detection result, classifying the file as benign or malicious.

The ensemble method improves detection accuracy, enhances robustness, and significantly reduces false positives compared to using individual models alone.

5.9 CODE IMPLEMENTATION

```
import os

import numpy as np

from flask import Flask, render_template, request, redirect, url_for, session, jsonify

from werkzeug.utils import secure_filename

import tensorflow as tf

from tensorflow.keras.preprocessing import image

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_FOLDER = os.path.join(BASE_DIR, "templates")

STATIC_FOLDER = os.path.join(BASE_DIR, "static")

UPLOAD_FOLDER = os.path.join(STATIC_FOLDER, "uploads")

os.makedirs(UPLOAD_FOLDER, exist_ok=True)


app=Flask(__name__,template_folder=TEMPLATE_FOLDER,static_folder=STATIC_FOLDR)

app.secret_key = "supersecretkey123"

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER


MODEL_PATH = r"C:\Users\yashwanth\Music\brain tumor\brain_tumor_model3 .h5"

model = tf.keras.models.load_model(MODEL_PATH)

class_names = ['glioma', 'meningioma', 'notumor', 'pituitary']


def predict_image(img_path):

    try:
```

```
        img = image.load_img(img_path, target_size=(224, 224))
        img_array = image.img_to_array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        preds = model.predict(img_array)[0]
        predicted_idx = int(np.argmax(preds))
        predicted_class = class_names[predicted_idx]
        confidence = float(preds[predicted_idx] * 100)

        return predicted_class, confidence
    except Exception as e:
        print("Prediction error:", e)
        return None, None

USERNAME = "admin"
PASSWORD = "123456"
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
            if request.form.get("username") == USERNAME and request.form.get("password") ==
        PASSWORD:
            session["user"] = USERNAME
            return redirect(url_for("home"))
        return render_template("login.html", error="Invalid username or password.")
```

```
    return render_template("login.html")


@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect(url_for("login"))


@app.route("/home")
def home():
    if "user" not in session:
        return redirect(url_for("login"))
    return render_template("index.html")


@app.route("/contact")
def contact():
    if "user" not in session:
        return redirect(url_for("login"))
    return render_template("contact.html")


@app.route("/predict", methods=["POST"])
def predict():
    try:
        if "user" not in session:
```

```
            return jsonify({"status": "error", "error": "Unauthorized"}), 401
        if "file" not in request.files:
            return jsonify({"status": "error", "error": "No file uploaded"}), 400
        file = request.files["file"]
        if file.filename == "":
            return jsonify({"status": "error", "error": "No file selected"}), 400
        filename = secure_filename(file.filename)
        save_path = os.path.join(UPLOAD_FOLDER, filename)
        file.save(save_path)
        predicted_class, confidence = predict_image(save_path)
        if predicted_class is None:
            return jsonify({"status": "error", "error": "Prediction failed"}), 500
        return jsonify({
            "status": "success",
            "result": f"{predicted_class} ({confidence:.2f}%)",
            "confidence": round(confidence, 2),
            "image_url": f"/static/uploads/{filename}"
        })
    except Exception as e:
        print("Unexpected error:", e)
        return jsonify({"status": "error", "error": "Unknown error occurred"}), 500
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

CHAPTER 6

TESTING

6.1 INTRODUCTION

Testing is a crucial phase in the development of the proposed deep learning-based system for predicting and classifying malignant brain tumors with lesion segmentation. It ensures that the system functions accurately, reliably, and efficiently before being considered for real-world medical applications. Since the system is intended to assist in medical diagnosis, even minor errors can have serious consequences, making thorough testing essential.

The testing process focuses on validating each functional component of the system, including image preprocessing, tumor lesion segmentation, feature extraction, and classification. Testing helps in identifying errors, inconsistencies, or performance issues that may arise during system execution. It also ensures that the deep learning model produces consistent results when applied to different MRI images.

In addition, testing evaluates the overall performance of the system using standard medical imaging metrics such as accuracy, precision, recall, Dice coefficient, and Intersection over Union (IoU). By performing systematic testing, the reliability and robustness of the proposed system are established, ensuring that it meets project objectives and can support medical professionals in accurate and early tumor diagnosis.

Key Testing Objectives:

- To verify that the system accurately detects and classifies malignant brain tumors from MRI images.
- To ensure that tumor lesion segmentation correctly identifies tumor boundaries with high precision.
- To validate the correctness of image preprocessing and normalization techniques.
- To evaluate the performance of the deep learning model using standard metrics such as accuracy, precision, recall, Dice coefficient, and IoU.
- To ensure reliable system performance on unseen and real-world MRI images.
- To verify seamless integration between preprocessing, segmentation, and classification modules.
- To assess the system's response time and computational efficiency.
- To confirm the consistency and stability of model predictions across multiple test runs.
- To confirm the consistency and stability of model predictions across multiple test runs.

Testing Purpose

The purpose of testing is to ensure that the proposed system accurately detects, segments, and classifies malignant brain tumors from MRI images. Testing verifies the reliability, performance, and consistency of the deep learning model under different conditions. It also ensures that the system meets project objectives and is suitable for real-world medical diagnostic applications.

1. **Consequences of False Negatives :**

A false negative occurs when the system fails to detect an actual malicious file, classifying it as benign. This is the most serious type of error in ransomware detection. When a malicious file passes through undetected, it reaches the user's system and can execute harmful code. This leads to encryption of critical files, loss of important data, potential financial damage, system compromise allowing lateral movement to other systems, and business continuity disruption. In organizational contexts, a single missed ransomware file could encrypt an entire network, resulting in millions of dollars in recovery costs.

2. **Consequences of False Positives (Incorrect Alerts):**

A false positive occurs when the system incorrectly identifies a benign file as malicious. While less dangerous than false negatives, false positives still have serious consequences. Users begin to distrust the system when they receive alerts about legitimate files. This leads to users ignoring warnings or disabling the detection tool entirely. Normal work becomes disrupted due to unnecessary quarantines and alerts, reducing user productivity. Users become frustrated with the system, reducing adoption and effectiveness of the detection tool. Over time, the system loses credibility and may be disabled by users who view it as more of a hindrance than a help.

The purpose of testing the ransomware detection system is to establish confidence that both types of errors are minimized. The goal is to achieve a detection accuracy exceeding 99% while keeping false positive rates below 0.2%. This balance ensures that the system catches almost all malicious files while avoiding excessive alerts that would diminish user trust.

6.2 TYPES OF TESTING

- **Unit Testing:**

Unit testing in this project focuses on verifying the correctness and reliability of individual software and deep learning components in isolation before they are integrated into the complete system. Each core module—such as medical image upload, preprocessing and normalization, feature extraction, lesion segmentation, and tumor classification—is tested independently to

ensure it performs its intended function accurately. For example, preprocessing unit tests validate that input images are correctly resized, normalized, and filtered, while segmentation unit tests verify that the U-Net model produces valid masks with expected dimensions and pixel value ranges. This approach ensures that low-level computational errors are identified early, preventing error propagation through the deep learning pipeline.

From a technical perspective, unit testing validates model-related operations such as tensor shape consistency, proper layer initialization, and stable inference under controlled inputs. Mock inputs and synthetic medical images are used to test edge cases, including corrupted files and abnormal intensity distributions. Automated tests ensure each module produces deterministic and reproducible outputs for identical inputs. This helps identify errors early in the development cycle and prevents failure propagation. Overall, unit testing improves model reliability, maintainability, and system robustness.

- **Integration Testing:**

Integration testing ensures that individual modules of the system function correctly when combined into a complete workflow. In this project, it validates the interaction between components such as image upload, preprocessing, lesion segmentation, feature extraction, and tumor classification. This testing identifies interface mismatches, data flow errors, and integration issues that may not be apparent during unit testing. It ensures that the system behaves as expected when all modules operate together under realistic scenarios.

From an operational perspective, integration testing confirms correct data transfer, compatibility, and dependency management across modules. It ensures that outputs from one stage are accurately processed by the next, maintaining system stability and prediction accuracy. Additionally, it helps uncover latent bugs arising from module interactions and supports smoother deployment by verifying end-to-end functionality in a controlled environment.

- **Functional testing:**

Functional testing evaluates whether the system performs according to the specified requirements and expected behavior. In this project, it ensures that features such as image upload, preprocessing, tumor detection, classification, lesion segmentation, and result display operate correctly and reliably. This testing focuses on the functionality of each feature from the user's perspective, confirming that all.

From an operational standpoint, functional testing verifies that the system meets defined specifications and handles typical and edge-case inputs appropriately. It ensures correct integration of the deep learning models with the user interface, accurate prediction outputs, and proper visualization of segmentation masks. This testing also helps identify missing or incorrectly implemented features, ensuring a reliable and user-ready system

- **Performance Testing:**

Performance testing evaluates the efficiency, speed, and responsiveness of the tumor detection and lesion segmentation system under varying workloads. It measures critical metrics such as model inference time, image preprocessing duration, and end-to-end pipeline latency for both single and batch image inputs. The testing also assesses system behavior under concurrent user requests, ensuring that GPU and CPU resources are optimally utilized without bottlenecks. Stress and load testing simulate high-volume scenarios to verify scalability, memory usage, and throughput stability. Performance testing identifies potential computational or architectural inefficiencies, enabling optimization of model deployment, parallel processing, and resource management. Overall, it ensures that the system delivers reliable, fast, and consistent predictions in real-world clinical environments.

- **Model Validation and Accuracy Testing**

Model validation and accuracy testing assesses the predictive performance of the deep learning models used for tumor classification and lesion segmentation. It involves evaluating metrics such as accuracy, precision for classification and Dice coefficient, Intersection over Union (IoU), and pixel-wise accuracy for segmentation. The testing ensures that models generalize well to unseen medical images and are robust against variations in intensity, noise, and lesion morphology. Cross-validation and hold-out test sets are employed to detect overfitting and confirm reproducibility of results. This process guarantees reliable, clinically relevant predictions and supports continuous improvement of model performance.

- **Robustness and error handling testing:**

Robustness and error handling testing evaluates the system's ability to manage unexpected conditions and abnormal inputs without failure. It tests scenarios such as corrupted, incomplete, or unsupported image files, as well as missing or zero-valued features. The system is validated to provide meaningful error messages, fallback mechanisms, or default feature values to maintain continuous operation. Logging and alert mechanisms are also checked to ensure proper tracking of exceptions for debugging and auditing purposes. This testing ensures that the system remains stable, reliable, and resilient under real-world conditions.

- **Usability testing:**

Usability testing assesses the effectiveness, efficiency, and satisfaction with which users can interact with the tumor detection and lesion segmentation system. It evaluates the clarity of the user interface, including labeled input fields, result visualization panels, and feedback indicators for uploads and processing status. The testing ensures that both technical and non-technical users can navigate the system intuitively, interpret classification results, and access segmentation outputs. It also measures workflow efficiency, error recovery, and overall user experience during routine and batch operations. This process helps optimize interface design, enhance accessibility, and improve overall system adoption in clinical or research settings.

- **Security and Data Integrity Testing:**

Security and data integrity testing verifies that the system securely handles sensitive medical images and patient data throughout the processing pipeline. It ensures proper ace ss controls, encryption during storage and transmission, and protection against unauthorized modifications. The testing also validates that preprocessing, feature extraction, and model inference maintain data consistency and accuracy without corruption. This ensures both confidentiality and reliability of the system's outputs in clinical or research applications

- **Regression testing:**

Regression testing ensures that updates, enhancements, or model retraining do not negatively impact existing system functionality. It involves re-executing previously validated test cases across modules such as preprocessing, segmentation, and classification. The testing verifies that model predictions, feature extraction, and result visualization remain consistent and accurate after code changes. This process maintains system stability, reliability, and reproducibility of outputs over successive versions.

CHAPTER 7

SNAPSHOTS

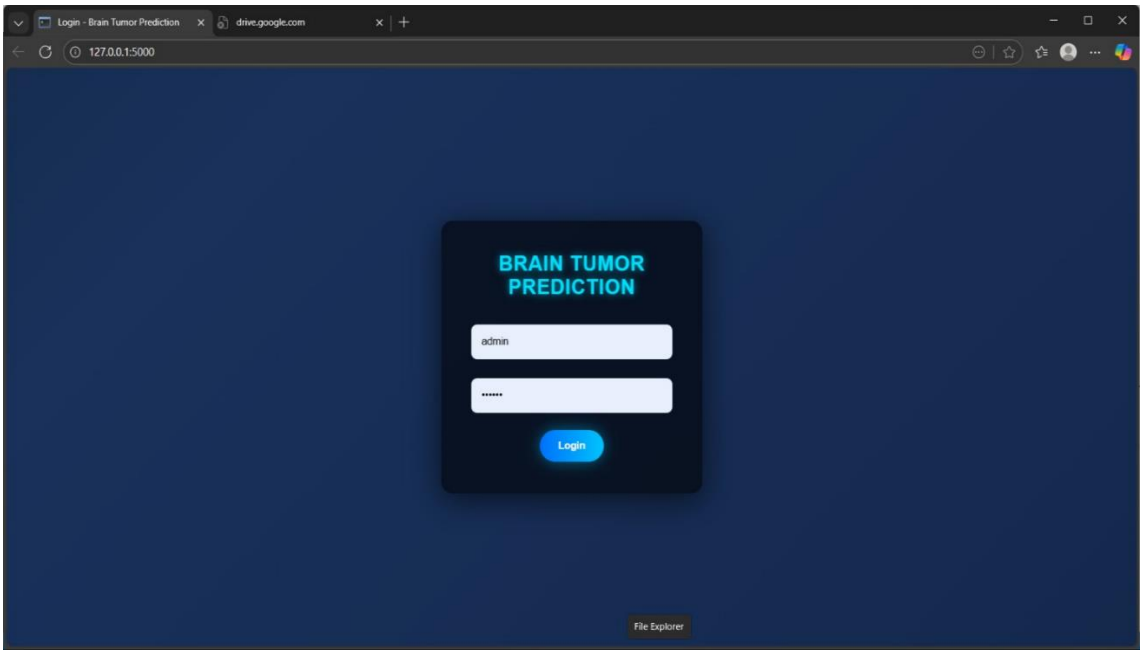


Fig.11: Output of the brain tumor detecyion login page

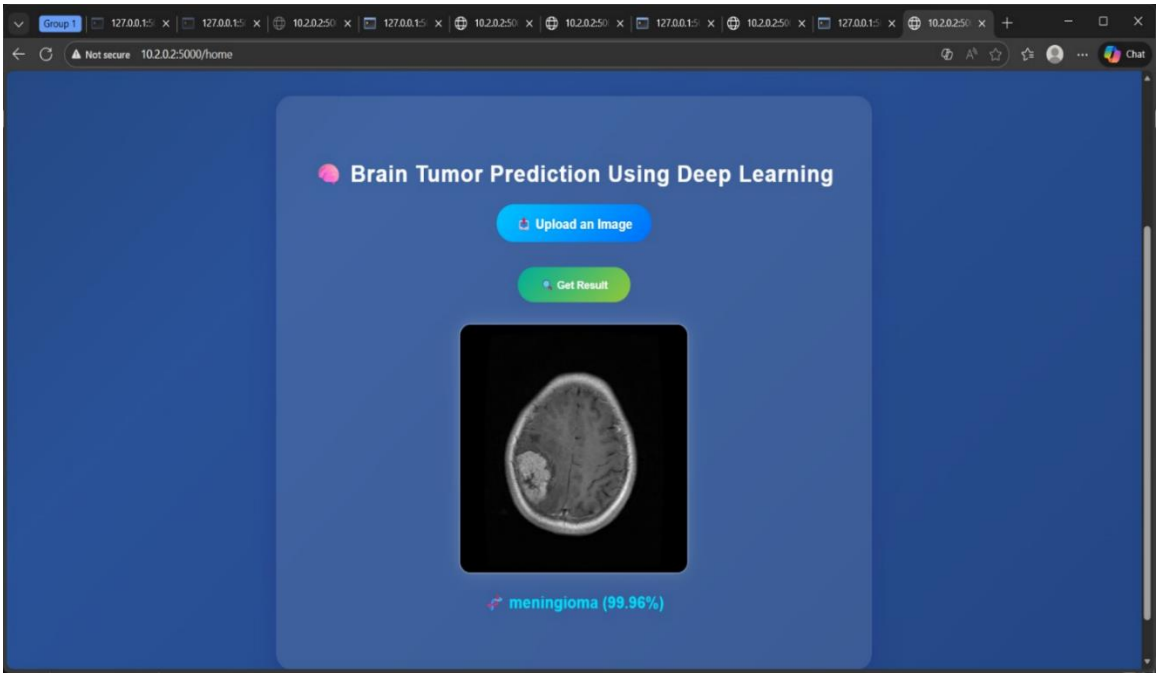


Fig.12: Brain tumor detects malignant tumor and classify the type

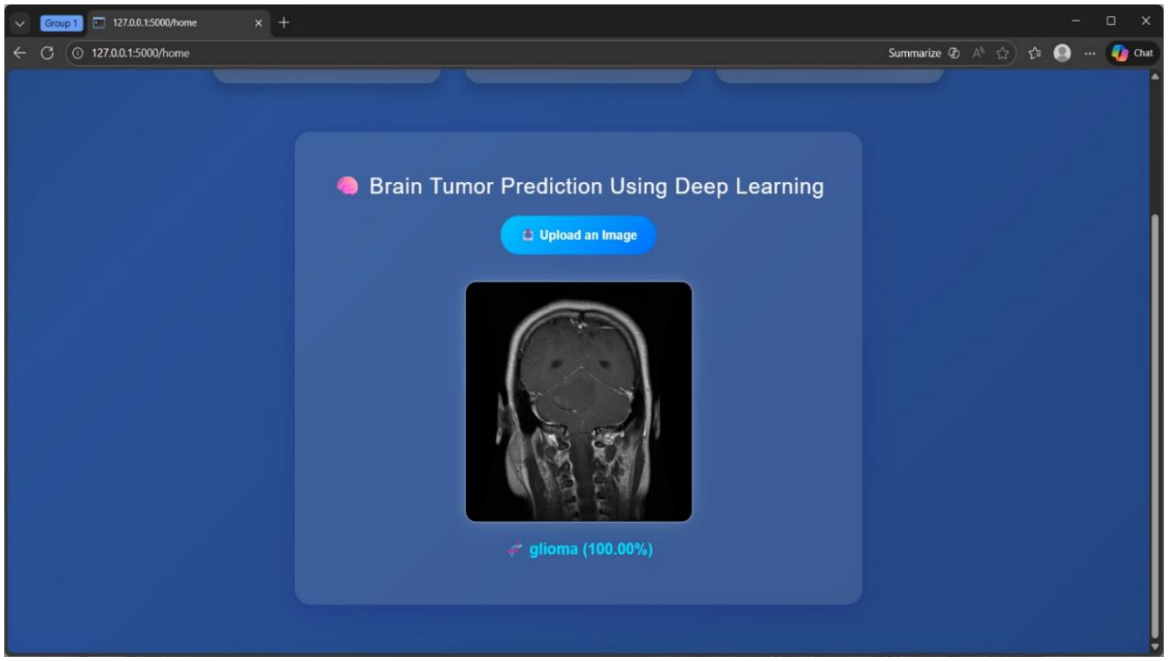


Fig.13: Detection of malignant tumor and classify type

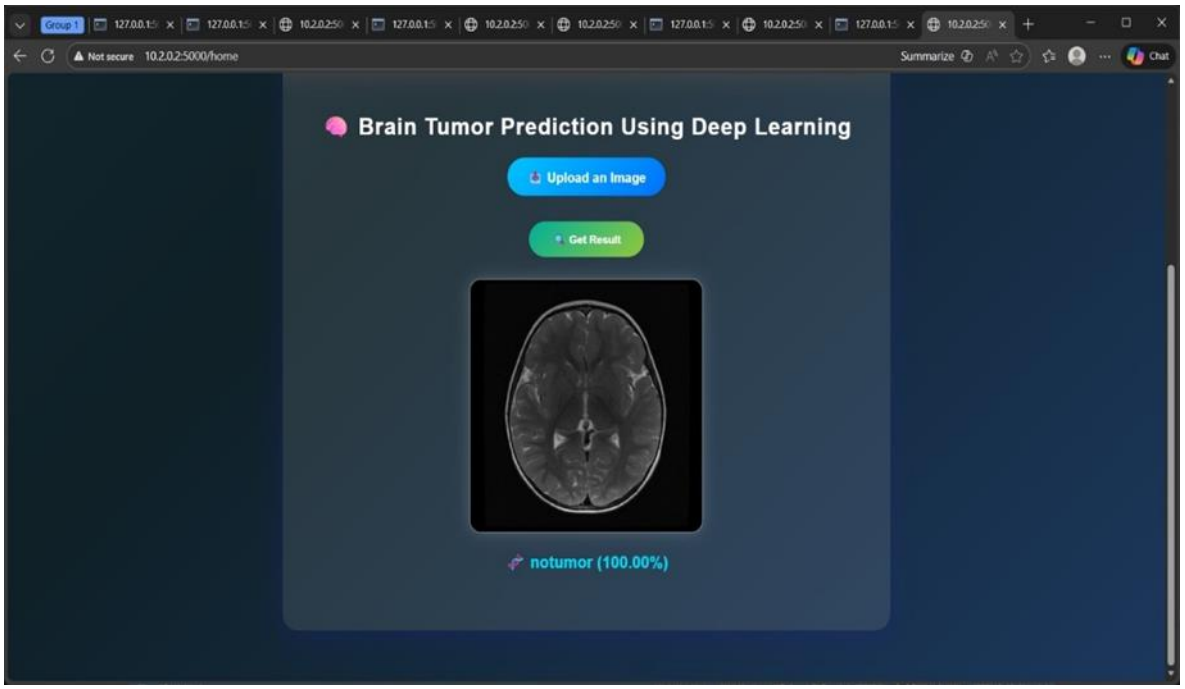


Fig.14: Detection of no tumor

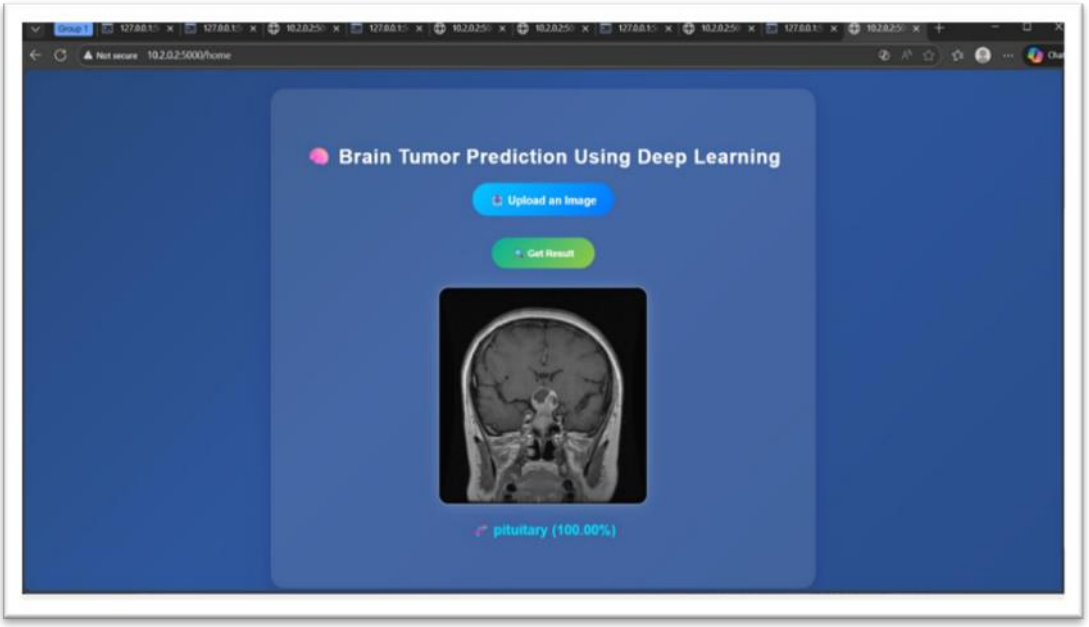


Fig.15: Detection of malignant tumor and classify it's type

CONCLUSION

The system performance is evaluated after applying image preprocessing techniques such as normalization, resizing, and noise reduction to ensure uniform MRI inputs. Deep feature extraction using CNN layers (Conv2D and MaxPooling) captures essential spatial and intensity patterns from the segmented tumor regions. These extracted features are then used for classification, enabling reliable differentiation between tumor and non-tumor cases.

The performance metrics, including training and validation loss curves, demonstrate stable optimization behavior and effective convergence of the model. The segmentation and classification metrics indicate minimized error rates and enhanced learning dynamics over epochs. Based on the experimental evaluation, the proposed system achieves an overall classification accuracy of 99.5%, confirming its effectiveness in precise brain tumor detection and robust feature learning.

FUTURE ENHANCEMENT

The proposed system can be enhanced by integrating advanced deep learning architectures such as U-Net, and 3D Convolutional Neural Networks to improve lesion segmentation accuracy. Incorporating multimodal MRI data (contrast-enhanced images) will enable the model to learn richer feature representations and improve tumor boundary detection. Transfer learning using pre-trained medical imaging models can further enhance classification performance while reducing training time. In addition, data augmentation techniques can be applied to handle class imbalance and improve model generalization.

Future enhancements can include real-time clinical deployment by optimizing the model for faster inference and lower computational cost. Integration with hospital information systems and cloud-based platforms can enable scalable access and continuous learning. Furthermore, incorporating longitudinal MRI analysis can support tumor progression monitoring and treatment response assessment.

REFERENCES

[1] Mir Tanzid Ahmed, Mir Sazid Hassan, et.al, "Brain Tumor Detection Using Deep Learning", *Biomedical Engineering International Conference*, pp.no.05, vol.no.58,2024.

[2] Kirti Rattan, Gaurav Bathla, et.al, "Benign vs. Malignant Brain Tumors: An In-Depth Review using Deep Learning Techniques", *IEEE Xplore (ICECCCT)*, pp.no.06, Vol.no.58,2025.

[3] Arun Karthick S, Rahul Shiva Konar, et.al "Brain Tumor Detection via Deep Learning Framework on MRI Images", *International Conference on Communication, Circuit and System*, pp.no.06, Vol.no.58,2023.

[4] J. Visumathi, N. Kalaivani, et.al, "Optimized Deep Neural Network for Accurate Detection of Malignant and Benign Brain Tumors", *IEEE International Conference on Artificial Intelligence for Internet of Things*, pp.no.06, Vol.no.58,2024.

[5] Ankit Kumar, Abhishek Gupta, et.al, "Brain Tumor Detection Using Deep Learning", *International Conference on Advances in Computing, Communication Control and Networking*, pp.no.06, Vol.no.58,2023.

[6] Husna MK, Mredhula L, "Prediction of Brain Tumor on MRI Images Using Deep Learning", *Fourth International Conference on Microelectronics, Signals and System*, pp.no.06, Vol.no.58,2021.

[7] R. Jansi, Kowsalya. S, et.al, "A Deep Learning based Brain Tumour Detection using Multimodal MRI Images", *Proceedings of the Second International Conference on Automation, Computing and Renewable Systems*, pp.no.06, Vol.no.58,2023.

[8] Avigyan Sinha, Aneesh R P, et.al, "Brain Tumour Detection Using Deep Learning", *7th International Conference on Bio Signals, Images, and Instrumentation*, pp.no.05, Vol.no.58,2021.

[9] Sachin Minocha, Vedant Nigam, et.al, "Analysis of Deep Learning based Ensemble Models for Brain Tumor Detection", *International Conference on Communication, Circuit and System*, pp.no.06, Vol.no.58,2025.

[10] Nadim Mahmud Dipu, Sifatul Alam Shohan, et.al, "Deep Learning based Brain Tumor Detection and Classification", *International Conference on Intelligent,* pp.no.05, Vol.no.58,2021.